

Capítulo 12

Inference Kv Cache

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 31/07/2026

Capítulo 12 — Inference I: KV-cache

Objetivo de aprendizagem: entender por que gerar texto é **desperdício** do jeito ingênuo, e construir o KV-cache — a otimização que torna a inferência de LLMs viável. E, junto com ela, aprender a distinção entre **prefill** e **decode**, que organiza tudo o que vem depois.

Pré-requisitos: Capítulos 4 (atenção), 5 (Transformer) e 11 (o modelo treinado).

Arquivos: - `modelo.py` — o GPT do Capítulo 11 com KV-cache, e o teste de equivalência - `gerar.py` — escreve texto usando o cache - `benchmark_cache.py` — quanto acelera e quanto de memória custa - `exercicios.md` — exercícios

Este capítulo não treina nada. Ele usa os pesos que o Capítulo 11 salvou em `modelo.pt`. Nenhum número de qualidade muda aqui — e é exatamente esse o ponto: a otimização precisa ser **invisível na saída** e visível só no relógio.

1. Um modelo que faz a mesma conta 128 vezes

Volte ao `gerar()` do Capítulo 11:

```
for _ in range(n_tokens):
    recorte = idx[:, -block_size:] # todo o contexto
    logits, _ = self(recorte) # forward COMPLETO
    logits = logits[:, -1, :] # ... e joga fora tudo menos a última posição
    ...
```

Leia de novo a terceira e a quarta linha. O modelo processa **todas** as posições do contexto e usa **uma**.

E na iteração seguinte ele faz tudo de novo — com o mesmo contexto de antes, mais um token. As posições 0 a 126 são recalculadas do zero, produzindo exatamente os mesmos números que produziram um instante atrás.

Para gerar 128 tokens com contexto 128, o modelo executa:

Caminho	Posições processadas
Trabalho útil	128 (uma por token gerado)
Trabalho realizado	$1 + 2 + 3 + \dots + 128 = 8.256$

Mais de 98% do cálculo é jogado fora. Não é uma ineficiência de implementação — é o algoritmo, escrito da forma mais direta possível.

E quanto isso vale no relógio? Menos do que você espera

Guarde este número antes de seguir, porque ele evita uma conclusão errada. Eliminando 98,4% do trabalho, o ganho medido neste modelo é de **1,93x** — não 60x.

Tokens gerados	Ingênuo	Com cache	Speedup
16	0,070 s	0,045 s	1,53x
128	0,906 s	0,469 s	1,93x

O motivo aparece na Seção 7: existe um **custo fixo por chamada** (~3,3 ms aqui) que os dois caminhos pagam igualmente, uma vez por token gerado. O cache elimina o trabalho *dentro* da chamada — que num modelo de 2,2 M parâmetros na CPU é a menor parte da conta.

Isso não torna o KV-cache dispensável; torna o **capítulo honesto**. A técnica é indispensável em escala real, onde o cálculo domina. Aqui ela é ensinável e o ganho é modesto. É a mesma lição do [Capítulo 8](#): otimizar o que não domina não acelera.

2. Por que dá para não repetir

A pergunta certa não é "como acelero a atenção?", e sim: **o que exatamente muda quando um token novo chega?**

Lembre do Capítulo 4. Para cada posição, a atenção calcula três vetores:

```
q = x @ Wq query : "o que eu procuro"
k = x @ Wk key  : "o que eu ofereço"
v = x @ Wv value : "o que eu entrego"
```

Repare que **k** e **v** da posição 3 dependem **só** do token na posição 3 e da sua posição. Não dependem de nada que venha depois — porque a máscara causal garante que a informação só flui do passado para o futuro.

Quando o token 4 chega, as chaves e valores das posições 0 a 3 são exatamente os mesmos de antes. Recalculá-los é repetir uma conta cujo resultado já temos.

Então guardamos. É isso o **KV-cache**: uma lista, por camada, com todos os **k** e **v** já calculados.

O que **não** dá para cachear é o **q**: a cada passo há uma query nova, a do token atual, e é ela que faz a pergunta ao passado inteiro.

O que sobra para calcular

Com o cache, gerar um token exige:

1. calcular **q**, **k**, **v** do token novo — uma posição, não T
2. concatenar o **k** e o **v** novos ao cache
3. atenção: uma query contra T chaves
4. feedforward, também de uma posição só

O custo por token deixa de crescer com o comprimento do texto já gerado. Na prática ele cresce um pouco — o passo 3 lê um cache cada vez maior — mas sai de "reprocessar tudo" para "ler o que está guardado".

3. A implementação: duas linhas

O bloco inteiro muda em duas linhas, dentro do `forward`:

```
if cache is not None:
    k_ant, v_ant = cache
    k = torch.cat((k_ant, k), dim=2) # dim=2 é a dimensão do tempo
    v = torch.cat((v_ant, v), dim=2)
```

O `k` e o `v` calculados nesta chamada valem só para o token novo. Concatenados ao que estava guardado, viram a sequência completa — e a atenção prossegue como sempre.

O formato do cache, por camada:

```
(B, n_head, T, head_size)
| | | +-- tamanho de cada cabeça (n_embd / n_head)
| | +----- quantos tokens já vimos <- só esta dimensão cresce
| +----- cabeças de atenção
+----- batch
```

Simples assim. **E é aqui que aparecem os dois erros que quase todo mundo comete.**

4. Armadilha 1 — a posição é absoluta

No modo *decode* passamos **um** token ao modelo. É tentador escrever:

```
x = self.te(idx) + self.pe(torch.arange(T)) # T == 1 -> posição 0
```

Errado. Esse token não está na posição 0 — está na posição *"quantos tokens já vieram antes"*. O embedding posicional precisa refletir isso:

```
ja_vistos = 0 if cache is None else cache[0][0].shape[2]
pos = torch.arange(ja_vistos, ja_vistos + T)
x = self.te(idx) + self.pe(pos)
```

Por que este bug é perigoso: ele **não quebra nada**. Não há exceção, não há `NaN`, não há erro de formato. O modelo continua gerando texto de aparência plausível, só que um pouco pior — porque todo token acha que é o primeiro da frase. Você só descobre comparando com a saída de referência.

5. Armadilha 2 — a máscara

A máscara causal impede que uma posição olhe para o futuro. No modo *decode*, quantas posições futuras existem? **Nenhuma** — o token atual é o último.

A única query é a do token novo, e ela **pode** olhar para todas as chaves do cache. Isso é precisamente o que a última linha da matriz triangular já dizia: toda de uns.

Aplicar `tril[:1, :T]` aqui seria um desastre: essa é a **primeira** linha da matriz, que só permite olhar a posição 0. O modelo passaria a enxergar apenas o primeiro token.

```
if T == T_total: # prefill: máscara normal
    w = w.masked_fill(self.tril[:T, :T] == 0, float("-inf"))
# decode (T == 1): nada a mascarar
```

Os dois erros têm a mesma assinatura: **degradam em silêncio**. É o motivo de a próxima seção existir.

6. O teste que define se está certo

Um KV-cache não é uma aproximação. Não é uma troca de qualidade por velocidade. É a **mesma conta**, reorganizada para não repetir trabalho.

Logo, o critério é binário:

Com a mesma semente, o texto gerado com cache tem de ser **idêntico**, token por token, ao gerado sem cache.

```
$ python modelo.py

ingenuo : [0, 310, 115, 10, 673, 521, 97, 496, 78, 656, 895, 702]
com cache : [0, 310, 115, 10, 673, 521, 97, 496, 78, 656, 895, 702]
idênticos : True
```

Se der **False**, não existe "quase certo": há um bug, e provavelmente é um dos dois das seções 4 e 5.

Uma nota sobre ponto flutuante. Aqui a igualdade é exata porque as operações acontecem na mesma ordem. Em modelos maiores, ou na GPU, a concatenação pode mudar a ordem de somas dentro de um kernel e produzir diferenças na última casa decimal. Nesse caso o critério vira "logits iguais dentro da tolerância de `float32`" — mas a exigência continua a mesma em espírito: a diferença tem de ser explicável por arredondamento, nunca por lógica.

7. Prefill e decode: duas fases, perfis opostos

Com o cache, a geração se separa naturalmente em duas fases — e essa distinção organiza tudo o que se faz em inferência de verdade.

Prefill processa o prompt inteiro de uma vez e monta o cache. É **uma** passada com T posições: uma *matmul* grande, que ocupa bem o processador.

Decode produz um token de cada vez. Para cada token, é preciso **ler todos os pesos do modelo** da memória — e usá-los para uma única posição.

Fase	Tokens por passada	Gargalo
Prefill	muitos	cálculo — matmuls grandes saturam as unidades
Decode	um	memória — lê todos os pesos para produzir 1 token

É a dicotomia latência/vazão do [Capítulo 8](#), reaparecendo num lugar diferente. E ela explica o comportamento que todo mundo já viu num chat: o modelo demora um instante para "começar" (prefill do seu prompt) e depois cospe palavras num ritmo constante (decode).

8. O preço: memória

O cache não é grátis. Ele guarda, **por camada**:

$$2 \text{ (K e V)} \times \text{batch} \times \text{n_head} \times T \times \text{head_size} \times \text{bytes_por_número}$$

Para o modelo deste curso é irrelevante. Para um modelo de verdade, não:

Modelo	Contexto	Batch	Cache
7B (32 camadas, 32 cabeças, hs 128), bf16	4.096	1	~2 GB
idem	4.096	32	~68 GB
idem	32.768	32	~550 GB

Os **pesos** desse modelo ocupam ~14 GB. Com batch 32 e contexto longo, o cache passa dos pesos — muitas vezes.

É por isso que servir um LLM é caro de um jeito que treinar não é. O cache cresce com o número de usuários simultâneos **vezes** o comprimento de cada conversa. Uma GPU que caberia o modelo inteiro não consegue atender 32 pessoas ao mesmo tempo.

Boa parte da engenharia de inferência moderna existe para atacar exatamente essa tabela:

Técnica	O que faz
Multi-query attention (MQA)	uma só cabeça de K/V para todas as de Q — divide o cache por n_head

Grouped-query attention (GQA)	meio-termo: grupos de cabeças compartilham K/V
Paged attention (vLLM)	aloca o cache em páginas, como memória virtual, sem desperdício
Quantização do cache	guarda K/V em 8 bits em vez de 16

O [Capítulo 13](#) trata da última.

9. O limite do contexto fixo

Este modelo tem embeddings posicionais aprendidos para 128 posições. Quando o texto passa disso, é preciso descartar o começo — e aí **as posições de todos os tokens mudam**, o que invalida o cache inteiro:

```
if cache[0][0].shape[2] >= self.block_size:
    logits, cache = self(idx[:, -self.block_size:]) # recomeça do zero
```

Toda vez que a janela desliza, você paga um prefill completo. É um custo real, e é consequência de uma escolha de arquitetura: **embeddings posicionais absolutos e aprendidos**.

As alternativas modernas (RoPE, ALiBi) codificam posição de forma **relativa**, e com elas a janela desliza sem invalidar o cache. Não é detalhe de otimização — é uma das razões pelas quais quase nenhum modelo atual usa o `nn.Embedding` posicional que construímos no Capítulo 4.

10. Resumo do capítulo

- Gerar texto do jeito ingênuo desperdiça **mais de 98%** do cálculo: cada token novo reprocessa todo o contexto anterior.
- As **chaves** e **valores** de uma posição não mudam quando tokens novos chegam — a máscara causal garante isso. Então dá para guardá-los.
- A implementação são **duas linhas** de concatenação, mais duas armadilhas que degradam em silêncio: a **posição absoluta** e a **máscara no decode**.
- O critério de correção é **saída idêntica**, não "parecida". Um cache que muda o texto está errado, não aproximado.
- **Prefill** é limitado por cálculo; **decode**, por memória. Reconhecer isso é o começo da engenharia de inferência.
- O cache custa **memória**, e em escala real ele **passa dos pesos** — o que faz da inferência um problema econômico diferente do treino.

Próximo capítulo

Capítulo 13 — Inference II: Quantization. Se o gargalo do decode é ler os pesos da memória, a saída óbvia é **encolher os pesos**. Vamos guardar o modelo em 8 bits — e medir o que isso custa em qualidade.

Exercícios — Capítulo 12 (KV-cache)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Estes exercícios usam o checkpoint do Capítulo 11 (`../11-datasets/modelo.pt`). Se ele não existir, rode antes `cd ../11-datasets && python train_text.py`.

E1 — O que dá e o que não dá para cachear (aquecimento)

Sem rodar: 1. Por que **k** e **v** de uma posição não mudam quando um token novo chega, mas **q** precisa ser recalculado a cada passo? 2. Se o modelo **não** tivesse máscara causal (atenção bidirecional, como no BERT), o KV-cache continuaria válido? Justifique. 3. Gerar 128 tokens com contexto 128 processa quantas posições no caminho ingênuo? E no caminho com cache? Qual a fração de trabalho descartado no primeiro?

E2 — Quebre a posição de propósito (importante)

No `forward` do `modelo.py`, troque o cálculo da posição por `torch.arange(T)` — o bug descrito na Seção 4 da apostila. 1. O programa levanta alguma exceção? O texto continua saindo? 2. Compare com a saída de referência (`gerar_ingenuo`). A partir de qual token elas divergem, e por quê **daquele** token em diante? 3. Leia os dois textos. Se você não tivesse a referência para comparar, conseguiria dizer qual está errado? O que isso diz sobre depurar otimizações de inferência?

E3 — Quebre a máscara de propósito (importante)

Agora aplique a máscara também no modo decode: troque a condição por `w = w.masked_fill(self.tril[:T, :T_total] == 0, float("-inf"))`, sem a guarda `if T == T_total`. 1. O que acontece com o texto? Descreva o padrão que aparece. 2. Explique a partir da matriz triangular: qual linha `tril[:1, :T_total]` seleciona, e o que ela permite o token enxergar? 3. Este bug é mais fácil ou mais difícil de notar que o do E2? Por quê?

E4 — Meça o ganho

Rode o `benchmark_cache.py` com a máquina ociosa. 1. Como o **ms por token** evolui com o comprimento, nos dois caminhos? Faça o gráfico mentalmente antes de olhar a tabela. 2. O speedup cresce, satura ou cai conforme o texto fica mais longo? Por quê? 3. Para **um único** token gerado, o cache ajuda? Existe comprimento em que ele atrapalha?

E5 — A conta de memória

1. Escreva a fórmula do tamanho do cache em função de `n_layer`, `n_head`, `head_size`, `T`, `batch` e bytes por número. Confira contra `bytes_do_cache()`.

2. Para um modelo de 7B (32 camadas, 32 cabeças, `head_size` 128) em bf16, com contexto 8.192: quanto ocupa o cache por usuário? E quantos usuários simultâneos cabem numa placa de 80 GB, descontando os ~14 GB dos pesos?
3. O cache cresce com $n_layer \times n_head \times head_size$. Note que $n_head \times head_size = n_embd$. Reescreva a fórmula em função de `n_embd` — o que isso diz sobre qual decisão de arquitetura controla o custo de servir o modelo?
-

E6 — Cache com batch

O `gerar_com_cache` gera uma sequência por vez. Faça-o gerar `B` sequências em paralelo. 1. O que muda na forma do cache? E no código? 2. Meça: gerar 32 sequências de 64 tokens em paralelo custa quanto, comparado a 32 gerações separadas? 3. Onde está o problema quando as sequências têm **comprimentos diferentes** (umas terminam antes)? Descreva pelo menos uma estratégia para lidar com isso.

E7 — Multi-query attention (desafio)

A MQA usa **uma só** cabeça de K/V, compartilhada por todas as cabeças de Q. O cache cai por um fator de `n_head`. 1. Implemente. Dica: `k` e `v` passam a ter forma `(B, 1, T, hs)`, e o *broadcast* faz o resto na multiplicação com `q`. 2. Quanto o cache encolhe no modelo deste curso? E num 7B? 3. **Cuidado com a conclusão.** Você pode implementar a MQA sobre os pesos já treinados, mas o modelo foi treinado com 6 cabeças de K/V independentes. Medindo a loss antes e depois, o que você espera? O que isso diz sobre quando a MQA precisa entrar no projeto?